Avalia: A Big Data Lens on the Lisbon Real-Estate Market

Daniel Freire Mendes Danish Mujtaba Qureshi Gregor Stefan Sonntag

Big Data Technologies

github.com/danielfmendes/Avalia avalia-frontend.pages.dev

Avalia turns twelve years of Portuguese property listings, plus tourism, search, and macro signals, into parish-level intelligence and twelve-month forecasts for Lisbon.

1 Introduction

Idealista, Imovirtual, and Casa Sapo only show *current* listings; none surfaces parish-level (*freguesia*) price history or the macro-economic and behavioural signals that move the market.

User stories.

- As a *renter or buyer*, I want decade-long price history for my parish, to negotiate from evidence.
- As a *small landlord or agent*, I want year-on-year parish benchmarks, to price listings competitively.
- As a *municipal planner*, I want housing-pressure indicators correlated with tourism and search, to target interventions.

Why this is a Big Data project. The base dataset is 2.6 GB of raw listings (∼3 M transactions) from *arquivo.pt* via the open-data project *habitacao.net*, joined with Google Trends and PORDATA at weekly-to-annual cadences (Section 3). Volume, Variety, and Veracity are present from day one; Velocity and full Value are analysed in Section 5.

2 System architecture and reproducibility

Seven pages share a D3-Geo Lisbon map that drills district → municipality → parish, filterable by sale type and room count:

- *Market Overview*: district prices, KPIs, drill-down map.
- *AI Predictions*: 12-month per-region forecast with uncertainty bands.
- *Signals*: price overlaid on Google Trends, *dormidas*, earnings, construction.
- *Compare*: side-by-side, up to six regions.
- *Rooms*: T0–T4+ trends per typology.
- *Affordability*: budget slider ranking parishes in reach.
- *Time Machine*: monthly scrubber back to 2012.

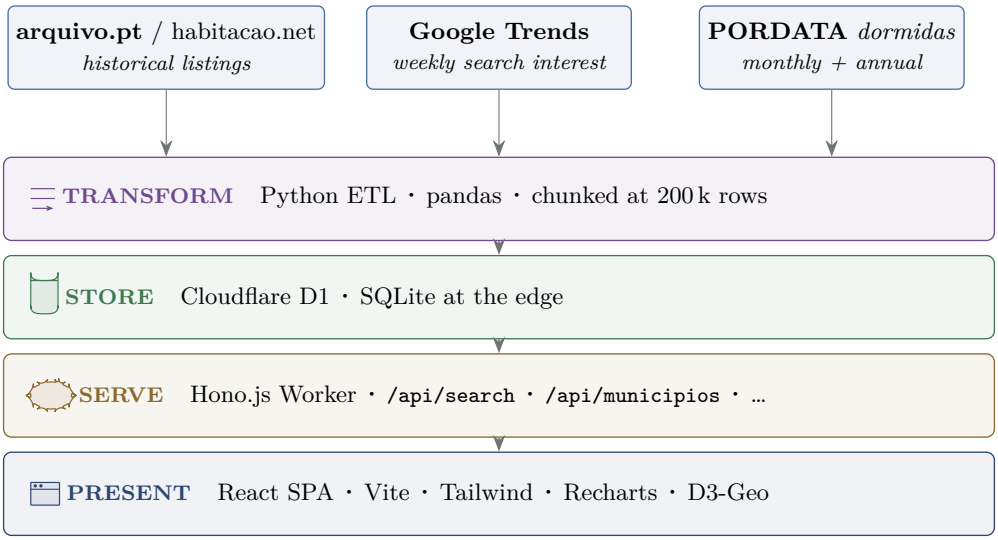

Figure 1: Avalia data pipeline. Three heterogeneous sources are normalised by a Python ETL into a single Cloudflare D1 schema, served by a Hono.js Worker, and consumed by a React single-page application with map-based drill-down.

The source code, ETL, and schema live in a **public GitHub repository**; the live demo is publicly accessible (References).

3 Data sources

- **arquivo.pt + habitacao.net.** *arquivo.pt* (Portugal's web archive) snapshots Idealista, Imovirtual, and Casa Sapo periodically; *habitacao.net* mines those snapshots to reconstruct listings history back to 2012 with price, area, rooms, and location per entry.
- **Google Trends.** Weekly Lisbon housing-keyword search interest, a leading indicator: spikes in queries like "comprar casa" precede price moves by one to two months.
- **PORDATA.** Per-municipality series: tourism overnights (*dormidas*, monthly) as a short-term-rental-pressure proxy, plus four annual macro series (foreign workers, tourist capacity, mean earnings, completed constructions).

Source	Content	Cadence
arquivo.pt + habitacao.net	2.6 GB raw listings, ∼3 M transactions	historical
Google Trends	Lisbon search interest	weekly
PORDATA *dormidas*	tourism overnights per municipality	monthly
PORDATA macro	foreign workers, tourist capacity, mean earnings, completed constructions	annual

Table 1: Data sources, contents, and refresh cadences.

4 Forecasting model

Per region, the frontend fits OLS to the trailing 18 monthly €/m^2 averages and projects 12 months ahead client-side, with the uncertainty band widening 1.5 % per step.

What is different. Where Idealista shows the present, Avalia shows *history* (*Time Machine* scrubs back to 2012) and *future* (12-month projections), and overlays exogenous signals on price

curves so users *see* what moves the market.

AI in the build. Code generation was AI-assisted (Claude Code); a representative prompt log is in Appendix A.

Planned upgrade. The current model extrapolates each region's own past prices and misses turning points; the next version adds Google Trends, *dormidas*, and the PORDATA macro series as exogenous predictors so the forecast reflects live demand and macro context.

5 The 5 Vs of Big Data: today and at scale

V	Today (Lisbon prototype)	At scale (national + live)
Volume	2.6 GB raw, aggregated to 65 k rows (Lisbon AML)	∼10 GB raw, ∼250 k aggregated, +0.5–1 M rows/yr live
Velocity	Batch reload only	Weekly *Idealista* snapshot, re-aggregated same day
Variety	Four external sources plus core listings	+ transit, school ratings, mortgage rates, weather, EU energy certificates
Veracity	Weighted means, chunked dedup, type coercion	Cross-portal reconciliation, anomaly detection, listing-survival modelling
Value	Yield calculator, affordability heatmap, signal overlay	Personalised recommendations, probabilistic forecasts, municipal-pressure index

Table 2: Mapping of the 5 Vs of Big Data onto the current prototype and the production-scale system.

Growth estimate. Portuguese real-estate transactions run at ∼150–200 k purchases per year. The forward plan is a **weekly *Idealista* snapshot**: every seven days the scraper pulls active listings per enabled region and the ETL re-aggregates them into the existing schema. Sizing from Lisbon's observed density (4.1 rows per parish per month, 88:1 raw → aggregated compression) and applying it across the other regions' parish counts (rural parishes are far sparser), the corpus grows from 65 k rows today to roughly **0.45 M after three years** and ∼0.6 M by 2030 – an order-of-magnitude expansion at constant Cloudflare D1 cost. Every gap in Table 2 is bridged by a known technology. Figure 2 traces the trajectory; dashed lines mark regional rollouts.

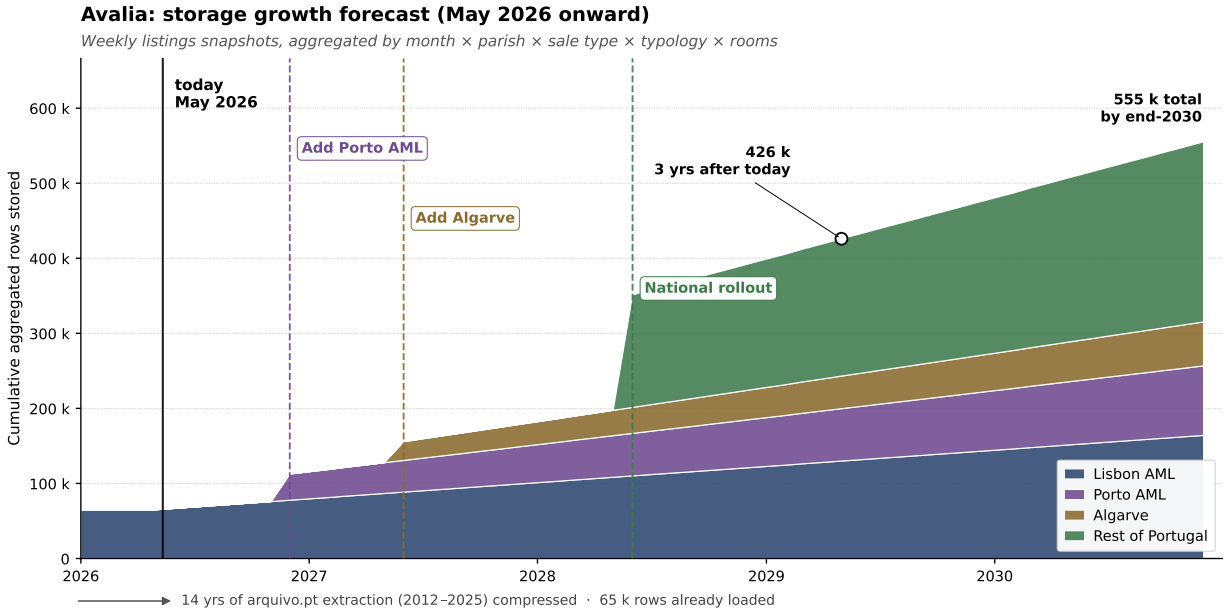

Figure 2: Cumulative aggregated rows stored, by region. The 14-year *arquivo.pt* extraction is compressed into the left-pointing annotation under the x-axis (65 k rows already loaded today); from May 2026 onward the forecast assumes a weekly *Idealista* snapshot per enabled region, aggregated into the existing schema. Per-region monthly row-rates derive from Lisbon's observed density of 4.1 rows per parish per month (measured from `summary_lisboa_final.csv`), multiplied by each region's parish count and boosted for weekly cadence; backfills on rollout use the same 88:1 raw → aggregated compression. All constants are documented at the top of `report/data_growth/data_growth.py`.

6 Discussion and conclusion

6.1 Monetisation and roadmap

Three revenue lines are plausible: a freemium consumer tier with a paid *investor* mode, a B2B API for banks and agencies, and anonymised market reports for municipalities. Next business steps: a paid validation pilot with one B2B partner (a small agency or appraisal firm) to test API willingness-to-pay, tiered usage-based pricing, and a GDPR / data-licensing review before any municipal contract.

6.2 Limitations

Cloudflare D1's free tier caps writes at **100 000 rows per day** (Cloudflare, 2025), which is why the prototype is scoped to the eighteen Lisbon AML municipalities; aggregation compresses ∼3 M raw rows to ∼65 k, inside the cap. The forecast does not yet use the exogenous regressors, and refresh is manual.

References

Cloudflare. (2025). *D1 Platform Limits*. Retrieved May 7, 2026, from https://developers.cloudflare.com/d1/platform/limits/

Fundação Francisco Manuel dos Santos. (n.d.). *PORDATA: Base de Dados Portugal Contemporâneo*. Retrieved May 7, 2026, from https://www.pordata.pt

Fundação para a Ciência e a Tecnologia. (n.d.). *Arquivo.pt: Portuguese Web Archive*. Retrieved May 7, 2026, from https://arquivo.pt

Google. (n.d.). *Google Trends*. Retrieved May 7, 2026, from https://trends.google.com

Habitação.net. (n.d.). *Dados sobre o mercado imobiliário em portugal*. Retrieved May 7, 2026, from https://www.habitacao.net/dados

Mendes, D. F., Qureshi, D. M., & Sonntag, G. S. (2026a). *Avalia (source code, public repository)*. https://github.com/danielfmendes/Avalia

Mendes, D. F., Qureshi, D. M., & Sonntag, G. S. (2026b). *Avalia live demo*. https://avalia-frontend.pages.dev

A AI prompt log

A curated, not verbatim, set of prompts that could reconstruct the project's current shape today, grouped by functional area and followed by a data-side problem we hit and the prompt that fixed it.

A.1 Data pipeline (Python ETL)

The pipeline is an offline, run-on-demand step that converts a single curated CSV into the SQL inserts that are then pushed to D1.

```
Write a small Python script that reads csv/summary_lisboa_final.csv and emits batched
INSERT INTO habitacao (...) statements into ../backend/sql/Insert_Queries.sql. The CSV
columns map 1:1 to the table columns: mes_ano, tipo_venda, tipo_habitacao, quartos,
distrito, municipio, freguesia, total_rows, avg_area, avg_preco, avg_m2. Group rows into
batches of 100 to avoid SQLITE_TOOBIG when loading into Cloudflare D1. Escape single
quotes by doubling them, leave numeric values unquoted, wrap text in single quotes.
```

A.2 Database schema (Cloudflare D1)

A single pre-aggregated table is enough, because aggregation has already been done in the ETL step.

```
Design a Cloudflare D1 schema for aggregated Portuguese housing listings. One table called
habitacao, columns: mes_ano (TEXT, "YYYY-MM"), tipo_venda, tipo_habitacao, quartos (TEXT,
numeric string like "0.0"), distrito, municipio, freguesia, total_rows (INTEGER, used as a
weight when re-aggregating), avg_area, avg_preco, avg_m2 (REAL). Add a composite index on
(mes_ano, municipio, freguesia) for the typical filter combination. Drop and recreate the
table on each load so the script is idempotent.
```

A.3 Public API (Hono Worker on Cloudflare)

```
Build a Hono Worker for Cloudflare that exposes four GET endpoints backed by a D1
binding called DB: /api/search (main query), /api/municipios (distinct municipalities),
/api/freguesias (parishes for a given municipio), /api/health. /api/search accepts
level=district|municipality|parish plus municipio, freguesia, tipo_venda, quartos
(T0..T4+, where T4+ means quartos >= 4), min_area, max_area. The database stores quartos
as a numeric string ("0.0","1.0",...), so normalise to T0..T4+ on the way out. Return JSON
{success, level, count, data}. CORS allowlist: env.ALLOWED_ORIGIN, http://localhost:5173,
http://127.0.0.1:3000.
```

A.4 Frontend dashboard (React + Tailwind + ShadcnUI)

Two prompts: the first lays out the seven-page skeleton, the second produces the Lisbon choropleth that ties the dashboard together.

```
Scaffold a Vite + React 19 + TypeScript + Tailwind v4 + ShadcnUI single-page application
with seven lazy-loaded routes: Market Overview, AI Predictions, Signals, Compare, Rooms,
Affordability, Time Machine. Add a top navigation bar that switches between them and a
dark/light toggle. Share filter state (sale type, room typology, area range, drill-down
selection) through a single DashboardContext provider that wraps the routed view. Use
Recharts for charts and the project alias @/ for src imports.
```

```
Write a React component LisbonMap that renders the 18 Lisbon Metropolitan Area
municipalities as a D3-Geo SVG choropleth. Fit a Mercator projection to the loaded GeoJSON
feature collection, fill each region by interpolating a 5-stop green-yellow-red colour
ramp over a price metric (EUR/m^2), show a hover tooltip with parish name, EUR/m^2, year-
on-year change, and listing count, and on click update the drill-down state so the map
re-renders showing only that municipality's parishes. Read the active metric and current
selection from the dashboard context.
```

A.5 Problem encountered: Lisboa-level aggregation

Early in the build the district-level view showed zero rows for Lisbon city. Inspecting the data, Lisboa turned out to be the only municipality whose pre-aggregated "Grouped at Municipio level" marker rows are absent from the source dataset; every other municipality has them, but for Lisboa only the underlying parish rows exist. A naive `WHERE freguesia = 'Grouped at Municipio level'` therefore filters Lisboa out entirely, leaving a blank tile on the district map. The fix synthesises Lisboa's municipality row on the fly at query time, weighted by `total_rows`. Cloudflare D1, which is SQLite-based and runs at the edge, does not handle correlated subqueries reliably for this shape, so the implementation uses `UNION ALL`. The resulting branch lives in `api/src/index.ts` under the `level=district` path.

```
In our D1-backed Hono Worker, GET /api/search?level=district returns no rows for Lisbon
city. Every other municipality has 'Grouped at Municipio level' rows in the habitacao
table, but Lisboa has only parish rows. Rewrite the SQL so the district response includes
Lisboa, computed on the fly as a weighted aggregate of its parish rows: averages are
SUM(metric * total_rows) / SUM(total_rows), and total_rows itself is summed. D1's
correlated-subquery support is unreliable, so combine the existing district rows with the
synthesised Lisboa row using UNION ALL. Group the synthesised side by mes_ano, tipo_venda,
tipo_habitacao, quartos, distrito, municipio. Order the final result by mes_ano ASC.
```

A.6 Secondary problem: parish-name mismatch on the Lisbon map

A related, smaller area-level issue: the choropleth had blank parishes for many of Lisboa's *freguesias*. The raw GeoJSON used parish names in slightly different casing, diacritics, and hyphen-versus-space conventions than the `freguesia` strings coming back from `/api/search` (for example *Avenidas Novas* versus *Avenidas-Novas*). The fix was to simplify the GeoJSON and route every name through a single normaliser before comparison.

```
In LisbonMap.tsx the choropleth fill is empty for several parishes because the names in
lisbon-parishes.json don't exactly match the freguesia strings returned by /api/search.
Both sources use Portuguese with diacritics and inconsistent hyphen-vs-space conventions.
Write a normalizeName helper that lowercases the input, strips diacritics, and replaces
every run of non-alphanumeric characters with a single hyphen; then key the fill lookup by
the normalised name on both sides so the map and the API agree.
```